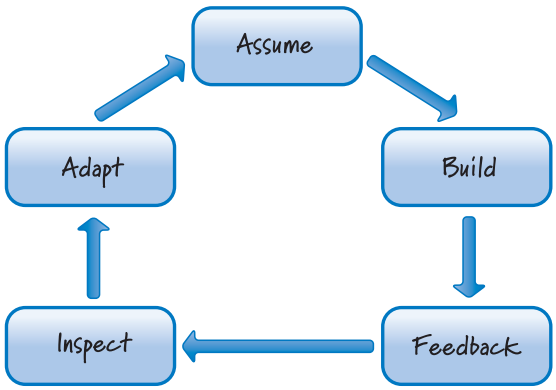


FIGURE 3.13 Learning loop pattern

The Scrum framework is also flexible enough to embrace many other learning loops. For example, although not specified by Scrum, technical practice feedback loops, such as pair programming (feedback in seconds) and test-driven development (feedback in minutes), are frequently used with Scrum development.

Organize Workflow for Fast Feedback

Being tolerant of long-lived assumptions also makes plan-driven processes tolerant of late learning, so **fast feedback** is not a focus. With Scrum, we strive for fast feedback, because it is critical for helping truncate wrong paths sooner and is vital for quickly uncovering and exploiting time-sensitive, emergent opportunities.

In a plan-driven development effort, every activity is planned to occur at an appointed time based on the well-defined phase sequence. This approach assumes that earlier activities can be completed without the feedback generated by later activities. As a result, there might be a long period of time between doing something and getting feedback on what we did (hence closing the learning loop).

Let's use component **integration** and testing as an example. Say we are developing three components in parallel. At some time these components have to be integrated and tested before we have a shippable product. Until we try to do the integration, we really don't know whether we have developed the components correctly. Attempting the integration will provide critical feedback on the component development work.

Using sequential development, integration and testing wouldn't happen until the predetermined downstream phase, where many or all components would be integrated. Unfortunately, the idea that we can develop a bunch of components in parallel and then later, in an integration phase, smoothly bring them together into a cohesive whole is unlikely to work out. In fact, even with well-conceived interfaces defined before we develop the components, it's likely that something will go wrong when we integrate them (see Figure 3.14).

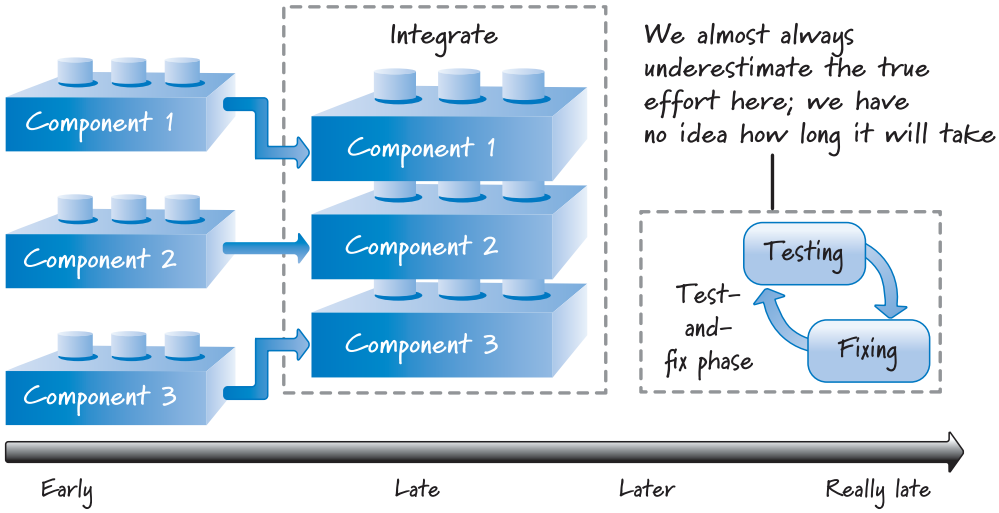


FIGURE 3.14 Component integration

Feedback-generating activities that occur a long time after development have unfortunate side effects, such as turning integration into a large test-and-fix phase, because components developed disjointedly from each other frequently don't integrate smoothly. How long it will take and how much it will cost to fix the problem can only be guessed at this point.

In Scrum, we organize the flow of work to move through the learning loop in Figure 3.13 and get to feedback as quickly as possible. In doing so, we ensure that feedback-generating activities occur in close time proximity to the original work. Fast feedback provides superior economic benefits because errors compound when we delay feedback, resulting in exponentially larger failures.

Let's look again at our component integration example. When we designed the components, we made important assumptions about how they would integrate. Based on those assumptions, we proceeded down a design path. We do not, at this point, know whether the selected design path is right or wrong. It's just our best guess.

Once we choose a path, however, we then make many other decisions that are based on that choice. The longer we wait to validate the original design assumption, the greater the number of dependent decisions. If we later determine (via feedback during the integration phase) that the original assumption was wrong, we'll have a large, compounded mess on our hands. Not only will we have many bad decisions that have to be reworked; we'll also have to do it after a great deal of time has passed. Because people's memories will have faded, they will spend time getting back up to speed on the work they did earlier.

When we factor in the total cost of reworking potentially bad dependent decisions, and the cost of the delay to the product, the economic benefits of fast feedback are very compelling. Fast feedback closes the learning loop quickly, allowing us to truncate bad development paths before they can cause serious economic damage.

Work in Process (WIP)

Work in process (or **WIP**) refers to the work that has been started but not yet finished. During product development WIP must be recognized and properly managed. I describe four agile principles related to this topic:

- Use economically sensible batch sizes.
- Recognize inventory and manage it for good flow.
- Focus on idle work, not idle workers.
- Consider cost of delay.

Use Economically Sensible Batch Sizes

Another core belief underlying plan-driven, sequential development processes is that it is preferable to batch up all of one type of work and perform it in a single phase. I refer to this as the **all-before-any** approach, where we complete all (or substantially all) of one activity before starting the next. Let's say we create all of the requirements during the analysis phase. Next, we move the batch of requirements into the design phase. Because we generated the complete set of requirements, our **batch size** in this example is 100%.

The all-before-any approach is, in part, a consequence of believing that the old manufacturing principle of economies of scale applies to product development. This principle states that the cost of producing a unit will go down as we increase the number of units (the batch size) that are produced. So, the sequential development belief is that larger batches in product development will also realize economies of scale.

In Scrum, we accept that although economies-of-scale thinking has been a bedrock principle in manufacturing, applying it dogmatically to product development will cause significant economic harm.

As counterintuitive as it might sound, working in smaller batches during product development has many benefits. Reinertsen discusses batch-size issues in depth, and Table 3.2 includes a subset of the small-batch-size benefits that he describes (Reinertsen 2009b).

If small batches are better than large batches, shouldn't we just use a batch size of one, meaning that we work on only one requirement at a time and flow it through all of the activities until it is done and ready for a customer? Some people refer to this as **single-piece flow**. As I will show in later chapters, a batch size of one might be appropriate in some cases, but assuming that "one" is the goal might suboptimize the flow and our overall economics.

TABLE 3.2 Reinertsen Benefits of Small Batch Sizes

Benefit	Description
Reduced cycle time	Smaller batches yield smaller amounts of work waiting to be processed, which in turn means less time waiting for the work to get done. So, we get things done faster.
Reduced flow variability	Think of a restaurant where small parties come and go (they flow nicely through the restaurant). Now imagine a large tour bus (large batch) unloading and the effect that it has on the flow in the restaurant.
Accelerated feedback	Small batches accelerate fast feedback, making the consequences of a mistake smaller.
Reduced risk	Small batches represent less inventory that is subject to change. Smaller batches are also less likely to fail (there is a greater risk that a failure will occur with ten pieces of work than with five).
Reduced overhead	There is overhead in managing large batches—for example, maintaining a list of 3,000 work items requires more effort than a list of 30.
Increased motivation and urgency	Small batches provide focus and a sense of responsibility. It is much easier to understand the effect of delays and failure when dealing with small versus large batches.
Reduced cost and schedule growth	When we're wrong on big batches, we are wrong in a big way with respect to cost and schedule. When we do things on a small scale, we won't be wrong by much.

Recognize Inventory and Manage It for Good Flow

Throughout this chapter, I have been reminding you that manufacturing and product development are not the same thing and thus should be approached differently. There is, however, one lesson that manufacturing has learned that we should apply to product development and yet often do not. That lesson has to do with the high cost of inventory, also known as work in process or WIP. The lean product development community has known for many years of the importance of WIP (Poppendieck and Poppendieck 2003; Reinertsen 2009b), and Scrum teams embrace this concept.

Manufacturers are acutely aware of their inventories and the financial implications of those inventories. How can they not be? Inventory quickly starts to pile up on the floor, waiting to be processed. Not only is factory inventory physically visible; it is also financially visible. Ask the CFO of a manufacturing company how much

inventory (or WIP) he has in the factory or how much it has changed in the past month and he can give a definitive answer.

No competent manufacturer sits on a large quantity of inventory. Parts that are sitting on the factory floor waiting to be put into finished goods are depreciating on the financial books. Worse yet, what happens if we purchase a truckload of parts, and then change the design of the product? What do we do with all of those parts? Maybe we rework the parts so that they fit into the new design. Or worse, maybe we discard the parts because they can no longer be used. Or, to avoid incurring waste on the parts we already purchased, are we going to not change our design (even though doing so would be the correct design choice) so we can use those parts—at the risk of producing a less satisfying product?

It's obvious that if we sit on a lot of inventory and then something changes, we experience one or more forms of significant waste. To minimize risks, competent manufacturers manage inventory in an economically sensible way—they keep some inventory on hand but use a healthy dose of just-in-time inventory management.

Product development organizations, generally speaking, are not nearly as cognizant of their work in process. Part of the problem stems from the fact that in product development we deal with knowledge assets that aren't physically visible in the same way as parts on the factory floor. Knowledge assets are far less intrusive, such as code on a disk, a document in a file cabinet, or a visual board on the wall.

Inventory in product development is also typically not financially visible. Ask a CFO of a product development organization how much inventory exists in the product development organization and he will likely give you a puzzled look and say, "None." While the financial team is tracking other measures of a product development effort, it won't likely be tracking product development inventory of this type.

Unfortunately, inventory (WIP) is a critical variable to be managed during product development, and the traditional approaches to product development don't focus on managing it. As I mentioned in the discussion of batch sizes, by setting the batch size to be quite large (frequently 100%), traditional development actually favors the creation of large amounts of inventory. An important consequence of having a lot of WIP in product development is that it significantly affects the cost-of-change curve I described earlier (see Figure 3.9).

Although we need some requirements if we are going to start development, we don't need to have *all* of the requirements. If we have too many requirements, we will likely experience inventory waste when requirements change. On the other hand, if we don't have enough requirements inventory, we will disrupt the fast flow of work, which is also a form of waste. In Scrum, our goal is to find the proper balance between just enough inventory and too much inventory.

It is important to realize that requirements are just one form of inventory that exists in product development. There are many different places and times during product development where we have WIP. We need to proactively identify and manage those as well.

Focus on Idle Work, Not Idle Workers

In Scrum, we believe that idle work is far more wasteful and economically damaging than idle workers. **Idle work** is work that we want to do (such as building or testing something) but can't do because something is preventing us. Perhaps we are blocked waiting on another team to do something, and until that team completes its work, we can't do ours. Or maybe we just have so much work to do that it can't all be done at once. In this case, some of the work sits idle until we become available to work on it. **Idle workers**, on the other hand, are people who have available **capacity** to do more work because they are not currently 100% utilized.

Many product development organizations focus more on eliminating the waste of idle workers than on the waste of idle work. For example, in traditional thinking, if I hire you to be a tester, I expect you to spend 100% of your time testing. If you spend less than 100% of your time testing, I incur waste (you're idle when you could be testing). To avoid this problem, I will find you more testing work to do—perhaps by assigning you to multiple projects—to get your utilization up to 100%.

Unfortunately, this approach reduces one form of waste (idle-worker waste) while simultaneously increasing another form of waste (idle-work waste). And, most of the time, the cost of the idle work is far greater than the cost of an idle worker. Let's explore why this is true.

To illustrate the issue, let's apply the keep-workers-100%-busy strategy to the 4×100-meter relay race at the Olympics. Based on the keep-them-busy strategy, this race seems highly inefficient. I pay people to run and they seem to be running only one-quarter of the time. The rest of the time they are just standing around. Well, that's not right! I pay them 100% salary so I want them to run 100% of the time. How about if when they're not carrying the baton, they just run up and down the stands or perhaps run another race on an adjacent track? That way they will be utilized 100% at running.

Of course, we all know that you don't win the relay gold medal by keeping the runners 100% busy. You win the gold medal by getting the baton across the finish line first. So, the important takeaway is "Watch the baton, not the runners" (Larman and Vodde 2009). In the context of product development, the baton sitting on the ground equates to work that is ready to be performed but is blocked waiting for necessary resources. You don't win the race (deliver products) when the baton is on the ground. (I really like the baton and runner analogy because it nicely illustrates that we should watch the work and not the workers. However, like any analogy, it has its limits. In this case, the relay-race approach to handing off work is precisely one aspect of traditional, sequential product development that we would like to avoid!)

Also, everyone knows the consequences of keeping a resource 100% busy. If we borrow a graph from queuing theory, we can see the obvious damage caused when striving for 100% utilization (see Figure 3.15).

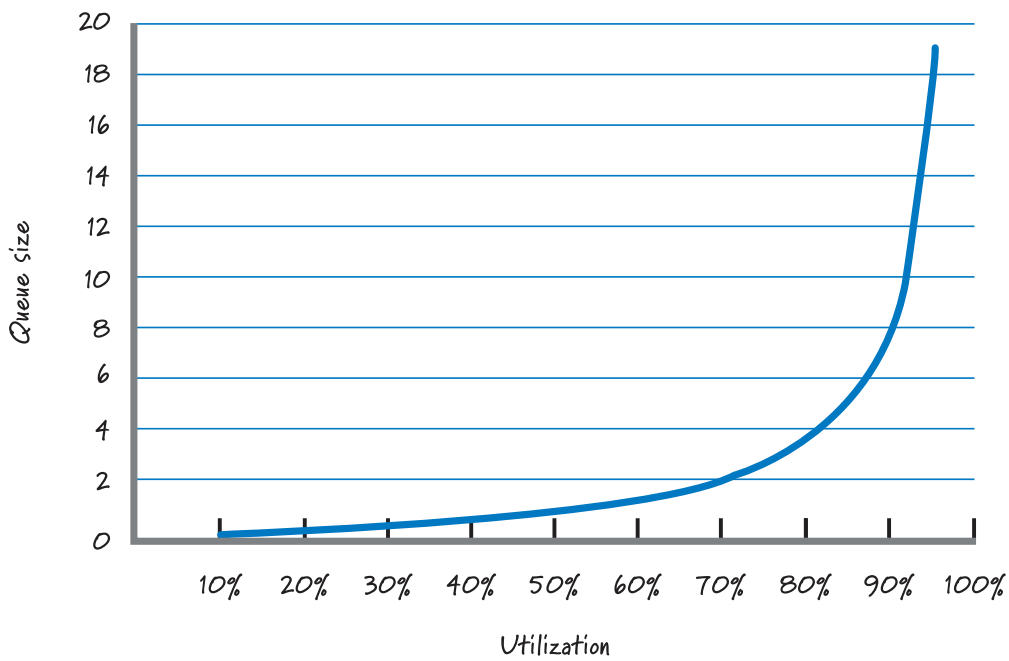


FIGURE 3.15 How utilization affects queue size (delay)

Anyone who owns a computer understands this graph. What happens if you run your computer at 100% (full processor and memory utilization)? It starts to thrash and every job on the computer slows down. In other words, the computer is working on more things and actually gets less productive work completed. Once you get into that state, it is very difficult to get out of it (you probably have to start killing jobs or reboot the machine). Your computer would be much more efficient if you ran it at closer to 80% utilization. In Figure 3.15, **queue** size equates to delay and delay to the baton sitting on the ground.

The idle work (delayed work) grows exponentially once we get into the high levels of utilization. And that idle work can be very expensive, frequently many times more expensive than the cost of idle workers (see the next section on cost of delay for an example). So, in Scrum, we are acutely aware that finding the bottlenecks in the flow of work and focusing our efforts on eliminating them is a far more economically sensible activity than trying to keep everyone 100% busy.

Consider Cost of Delay

Cost of delay is the financial cost associated with delaying work or delaying achievement of a milestone. Figure 3.15 illustrates that as capacity utilization increases, queue size and delay also increase. Therefore, by reducing the waste of idle workers (by increasing their utilization), we simultaneously increase the waste associated

with idle work (work sitting in queues waiting to be serviced). Using cost of delay, we can calculate which waste is more economically damaging.

Unfortunately, 85% of organizations don't quantify cost of delay (Reinertsen 2009b). Combine that with the fact that most development organizations don't realize they have accumulated work (inventory) sitting in queues, and it is easy to see why their default behavior is to focus on eliminating the visible waste of idle workers.

Here is a simple example to illustrate why the cost of idle work is typically much greater than the cost of idle workers. Consider this question: Should we assign a documenter to the team on the first day of development or at the end of development? Table 3.3 illustrates a comparison of these two options (there are other options we could use).

Assume that we assign the documenter full-time for 12 months to work on this product, even if he is not needed 100% of the time. Doing so costs an incremental $75K (think of this as idle worker waste) above what it would cost if we brought him on for two months at the end once the product reaches the state of "all but documented."

If we assign the documenter to do all of the documentation at the end, we will need him full-time for only two months, but we will also delay the delivery of the

TABLE 3.3 Example Cost-of-Delay Calculation

Parameter	Value
Duration with full-time documenter	12 months
Duration with documenter assigned at the end (when we reach the state of "all but documented")	14 months
Cycle-time cost for doing documentation at the end	2 months
Cost of delay, per month	$250K
Total cost of delay	**$500K**
Annual fully burdened cost of documenter	$90K
Monthly fully burdened cost of documenter	$7.5K
Cost for full-time documenter	$90K
Cost for documenter if assigned at end	$15K
Incremental cost for full-time documenter	**$75K**

product by the same two months. If we delay shipping the product by two months, the calculated cost of delay in terms of lifecycle profits is $500K (**lifecycle profits** are the total profit potential of a product over its lifetime; in this example, that potential decreases by $500K).

In this example, the cost of the idle worker is $75K and the cost of the idle work is $500K. If we focus on optimizing the utilization of the documenter, we will substantially suboptimize the economics of the overall product. During product development we are presented with these types of trade-offs on a continuous basis; cost of delay will be one of the most important variables to consider when making economically sensible decisions.

Progress

When using Scrum, we measure progress by what we have delivered and validated, not by how we are proceeding according to the predefined plan or how far we are into a particular phase or stage of development. I describe three agile principles related to this topic:

- Adapt to real-time information and replan.
- Measure progress by validating working assets.
- Focus on value-centric delivery.

Adapt to Real-Time Information and Replan

In a plan-driven, sequential process, the plan is the authoritative source on how and when work should occur. As such, conformance to the plan is expected. In contrast, in Scrum we believe that unbridled faith in the plan will frequently blind us to the fact that the plan might be wrong.

On a Scrum development effort our goal is not to conform to some plan, some up-front prediction of how we thought things might go. Instead, our goal is to rapidly replan and adapt to the stream of economically important information that is continuously arriving during the development effort.

Measure Progress by Validating Working Assets

Progress during a sequential, plan-driven development effort is demonstrated by completing a phase and being permitted to enter the next phase. As a result, if each phase starts and completes as expected, the product development effort might seem to be progressing quite well. Yet in the end, the product we created in full accordance with the plan might deliver far less customer value than anticipated. Can we really claim success if we finish on time and on budget and yet fail to meet customer expectations?

With Scrum, we measure progress by building working, validated assets that deliver value and that can be used to validate important assumptions. This gives us the feedback to know what the right next step is. In Scrum, it's not about how much work we start; it's all about what customer-valuable work we finish.

Focus on Value-Centric Delivery

Plan-driven, sequential development focuses on diligently following the process. By its very structure, the integration and delivery of features during sequential development happen at the end of the effort (see Figure 3.16). With this approach there is a risk that we will run out of resources (time or money) before we deliver all of the important value to our customers.

A related belief of traditional development is that the planning and document artifacts that get produced en route to delivering features are themselves valuable. If these artifacts are indeed valuable, most of the time they are valuable only to the downstream process and not the customers. And, if they are valuable to the customer, that value accrues only if a desirable product is ultimately delivered to the customer. Until that happens, these artifacts provide no direct customer value.

Scrum, on the other hand, is a customer-value-centric form of development. It is based on a prioritized, incremental model of delivery in which the highest-value features are continuously built and delivered in the next iteration. As a result, customers get a continuous flow of high-value features sooner.

In Scrum, value is generated by delivering working assets to customers, by validating important assumptions, or by acquiring valuable knowledge. In Scrum, we believe that the intermediate artifacts provide no perceived customer value and are merely a means to an end if they themselves cannot be used to generate important feedback or acquire important knowledge.

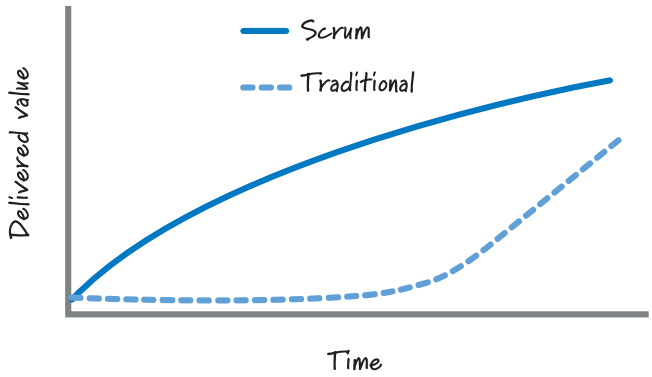


FIGURE 3.16 Deliver high-value features sooner.

Performance

There are specific performance-related characteristics we expect when using Scrum. I describe three agile principles related to this topic:

- Go fast but never hurry.
- Build in quality.
- Employ minimally sufficient ceremony.

Go Fast but Never Hurry

Plan-driven development believes that if we follow the plan and do things right the first time, we'll avoid costly and time-consuming rework. Moving from step to step quickly is of course desirable, but it isn't a principal goal.

In Scrum, one core goal is to be nimble, adaptable, and speedy. By going fast, we deliver fast, we get feedback fast, and we get value into the hands of our customers sooner. Learning and reacting quickly allow us to generate revenue and/or reduce costs sooner.

Do not, however, mistake going fast for being hurried. In Scrum, time is of the essence, but we don't rush to get things done. Doing so would likely violate the Scrum principle of **sustainable pace**—people should be able to work at a pace that they can continue for an extended period of time. In addition, hurrying will likely come at the expense of quality.

An example might help clarify the difference between fast and hurried. I study Muay Thai (Thai kickboxing). As is true of most martial arts, Muay Thai performance is enhanced with speed. Being able to swiftly and accurately perform katas or sparring enhances the pleasure of the sport and the outcome. However, hurrying through the movements with the intent of getting done substantially reduces their effectiveness and could cause serious bodily harm during sparring. When performing Muay Thai, you move swiftly, nimbly, and deliberately while quickly adapting to the situation. In other words, you need to be fast, but never hurried.

Build In Quality

During plan-driven development, the belief is that through careful, sequential performance of work we get a high-quality product. However, we can't actually verify this quality until we do the testing of the integrated product, which occurs during a late phase of the process. If testing should indicate that the quality is lacking, we then must enter the costly test-and-fix phase in an attempt to test quality in. Also, because a different team frequently works on each phase, the testing team is often viewed as owning the quality of the result.

In Scrum, quality isn't something a testing team "tests in" at the end; it is something that a cross-functional Scrum team owns and continuously builds in and

verifies every sprint. Each increment of value that is created is completed to a high level of confidence and has the potential to be put into production or shipped to customers (see Chapter 4 for a deeper discussion of the definition of done). As a result, the need for any significant late testing to tack on quality is substantially reduced.

Employ Minimally Sufficient Ceremony

Plan-driven processes tend to be high-**ceremony**, document-centric, process-heavy approaches. A side effect of Scrum's being value-centric is that very little emphasis is put on process-centric ceremonies. I don't mean to imply that all ceremony is bad. For example, a "ceremony" of going to the pub to socialize and bond every Friday after work would be a good ceremony. I am referring to ceremony that is **unnecessary formality**. Some might call it "process for the sake of process." Such ceremony has a cost but adds little or no value (in other words, it's a type of waste).

Example ceremonies that might be unnecessary formality include the following:

- A three-day, heavyweight process is required for approving and migrating code from the development environment to the QA environment before we are allowed to start testing.
- All anomalies have to be logged into a software tool so that they can be tracked and reported, even if I could just tap on the shoulder of the person sitting next to me and say, "Hey, this isn't working; could you fix it?" and have him make a fix so I can continue my work.
- I write a document because now is the prescribed time to write that document, even though nobody can say why that document is necessary or valuable.

In Scrum, our goal is to eliminate unnecessary formality. Therefore, we set the ceremonial bar at a low level, one that is minimally sufficient (some call it barely sufficient) or good enough. Of course, what constitutes minimally sufficient or good enough can differ from organization to organization. If we're building a new social media website, our need for ceremony might be exceptionally low. On the other hand, if we're building a pacemaker and are subject to numerous governmental regulations that require specific types of ceremonies, the minimally sufficient bar will be set higher (see Figure 3.17).

Frequently the Scrum focus on minimally sufficient ceremony is misinterpreted to mean things like "Scrum is anti-documentation." Scrum isn't anti-documentation. Rather, when using Scrum, we adopt an economic perspective and carefully review which documents we create. If we write a document that is shelfware and adds no value, we have wasted our time and money creating a dead document. However, not all documents are dead. For example, we will likely write a document if

- It is a deliverable as part of the product (for example, installation instructions, user's guide, and so on)

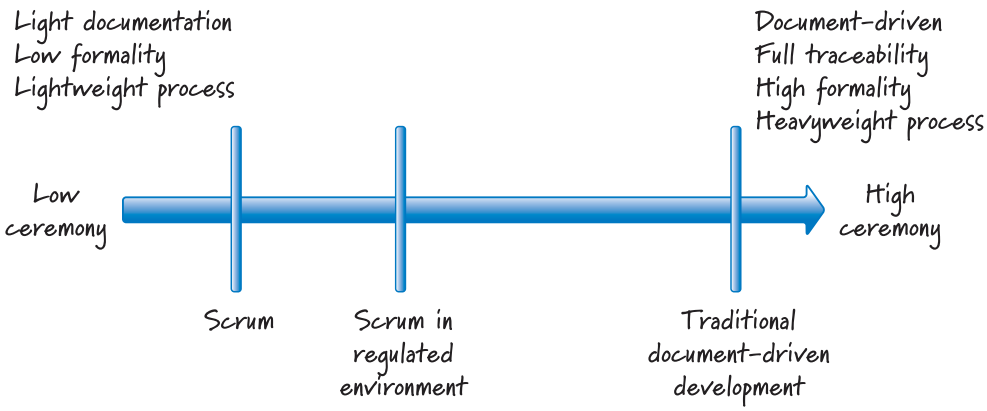


FIGURE 3.17 Ceremony scale

- Our goal is to capture an important discussion, decision, or agreement so that in the future we will have a clear recollection of what was discussed, decided, or agreed to
- It is the high-value way of helping new team members come up to speed quickly
- There is a regulatory requirement that a certain document be written (a cost of doing business in a regulated industry)

What we are trying to avoid is work that adds no short-term or long-term economic value. In Scrum, we believe that time and money are better spent delivering customer value.

Closing

In this chapter I focused on describing core agile principles—the fundamental beliefs that drive how we develop with Scrum. In doing so, I compared how these beliefs are different from the beliefs that underlie textbook, traditional, plan-driven, sequential development (which are summarized in Table 3.4).

My goal of making this comparison is not to convince you that waterfall is bad and that Scrum is good. Instead, my goal is to illustrate that the underlying beliefs of waterfall make it more appropriate to a different class of problem than Scrum. You can evaluate for yourself what type of problems your organization addresses and therefore which is the more appropriate tool to use. The subsequent chapters of this book will provide a detailed description of how these principles reinforce one another, providing a powerful approach to product development.

TABLE 3.4 Comparison Summary of Plan-Driven and Agile Principles

Topic	Plan-Driven Principle	Agile Principle
Similarity between development and manufacturing	Both follow a defined process.	Development isn't manufacturing; development creates the recipe for the product.
Process structure	Development is phase-based and sequential.	Development should be iterative and incremental.
Degree of process and product variability	Try to eliminate process and product variability.	Leverage variability through inspection, adaptation, and transparency.
Uncertainty management	Eliminate end uncertainty first, and then means uncertainty.	Reduce uncertainties simultaneously.
Decision making	Make each decision in its proper phase.	Keep options open.
Getting it right the first time	Assumes we have all of the correct information up front to create the requirements and plans.	We can't get it right up front.
Exploration versus exploitation	Exploit what is currently known and predict what isn't known.	Favor an adaptive, exploratory approach.
Change/emergence	Change is disruptive to plans and expensive, so it should be avoided.	Embrace change in an economically sensible way.
Predictive versus adaptive	The process is highly predictive.	Balance predictive up-front work with adaptive just-in-time work.
Assumptions (unvalidated knowledge)	The process is tolerant of long-lived assumptions.	Validate important assumptions fast.
Feedback	Critical learning occurs on one major analyze-design-code-test loop.	Leverage multiple concurrent learning loops.
Fast feedback	The process is tolerant of late learning.	Organize workflow for fast feedback.

continues

TABLE 3.4 Comparison Summary of Plan-Driven and Agile Principles (*Continued*)

Topic	Plan-Driven Principle	Agile Principle
Batch size (how much work is completed before the next activity can start)	Batches are large, frequently 100%—all before any. Economies of scale should apply.	Use smaller, economically sensible batch sizes.
Inventory/work in process (WIP)	Inventory isn't part of the belief system so is not a focus.	Recognize inventory and manage it to achieve good flow.
People versus work waste	Allocate people to achieve high levels of utilization.	Focus on idle work, not idle workers.
Cost of delay	Cost of delay is rarely considered.	Always consider cost of delay.
Conformance to plan	Conformance is considered a primary means of achieving a good result.	Adapt and replan rather than conform to a plan.
Progress	Demonstrate progress by progressing through stages or phases.	Measure progress by validating working assets.
Centricity	Process-centric—follow the process.	Value-centric—deliver the value.
Speed	Follow the process; do things right the first time and go fast.	Go fast but never hurry.
When we get high quality	Quality comes at the end, after an extensive test-and-fix phase.	Build quality in from the beginning.
Formality (ceremony)	Formality (well-defined procedures and checkpoints) is important to effective execution.	Employ minimally sufficient ceremony.

Chapter 4
SPRINTS

Scrum organizes work in iterations or cycles of up to a calendar month called sprints. This chapter provides a more detailed description of what sprints are. It then discusses several key characteristics of sprints: They are timeboxed, have a short and consistent duration, have a goal that shouldn't be altered once started, and must reach the end state specified by the team's definition of done.

Overview

Sprints are the skeleton of the Scrum framework (see Figure 4.1).

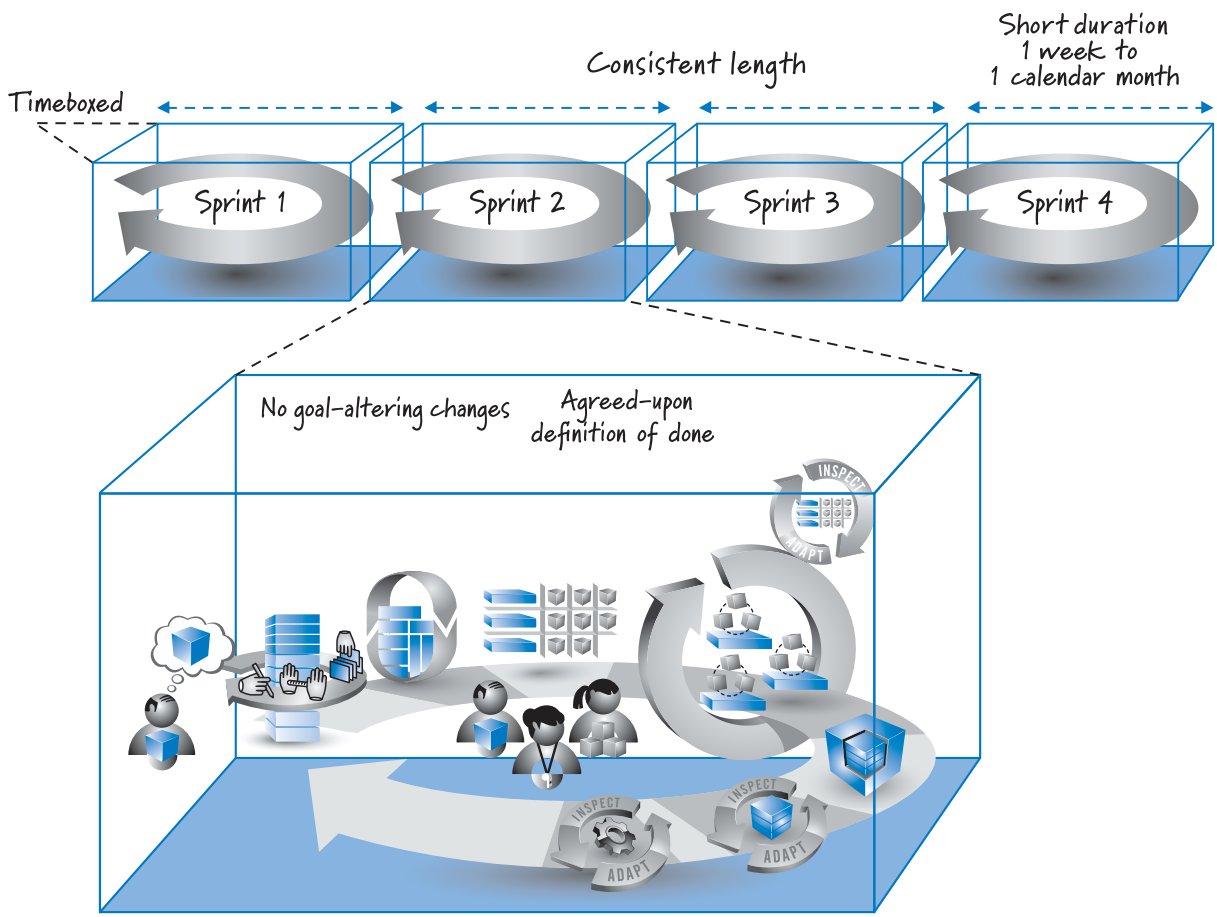


FIGURE 4.1 Sprints are the skeleton of the Scrum framework.

The gray main looping arrow in the figure, which stretches from the product backlog through the sprint execution loop and encompasses the Scrum team members, represents the sprint, on which the other Scrum artifacts and activities are shown oriented by their relative time of occurrence within the sprint. Although sprint execution is frequently confused with being "the sprint," it's really just one activity that occurs during the sprint, along with sprint planning, sprint review, and the sprint retrospective.

All sprints are timeboxed, meaning they have fixed start and end dates. Sprints must also be short, somewhere between one week and a calendar month in length. Sprints should be consistent in length, though exceptions are permitted under certain circumstances. As a rule, no goal-altering changes in scope or personnel are permitted during a sprint. Finally, during each sprint, a potentially shippable product increment is completed in conformance with the Scrum team's agreed-upon definition of done.

Although each organization will have its own unique implementation of Scrum, these sprint characteristics, with a few exceptions that we'll explore, are meant to apply to every sprint and every team. Let's look at each in detail so that we can understand why this is so.

Timeboxed

Sprints are rooted in the concept of **timeboxing**, a time-management technique that helps organize the performance of work and manage scope. Each sprint takes place in a time frame with specific start and end dates, called a timebox. Inside this timebox, the team is expected to work at a sustainable pace to complete a chosen set of work that aligns with a sprint goal.

Timeboxing is important for several reasons (see Figure 4.2).

Establishes a WIP Limit

Timeboxing is a technique for limiting the amount of WIP (work in process). WIP represents an inventory of work that is started but not yet finished. Failing to properly manage it can have serious economic consequences. Because the team will plan to work on only those items that it believes it can start and finish within the sprint, timeboxing establishes a WIP limit each sprint.

Forces Prioritization

Timeboxing forces us to prioritize and perform the small amount of work that matters most. This sharpens our focus on getting something valuable done quickly.

Demonstrates Progress

Timeboxing also helps us demonstrate relevant progress by completing and validating important pieces of work by a known date (the end of the sprint). This type of

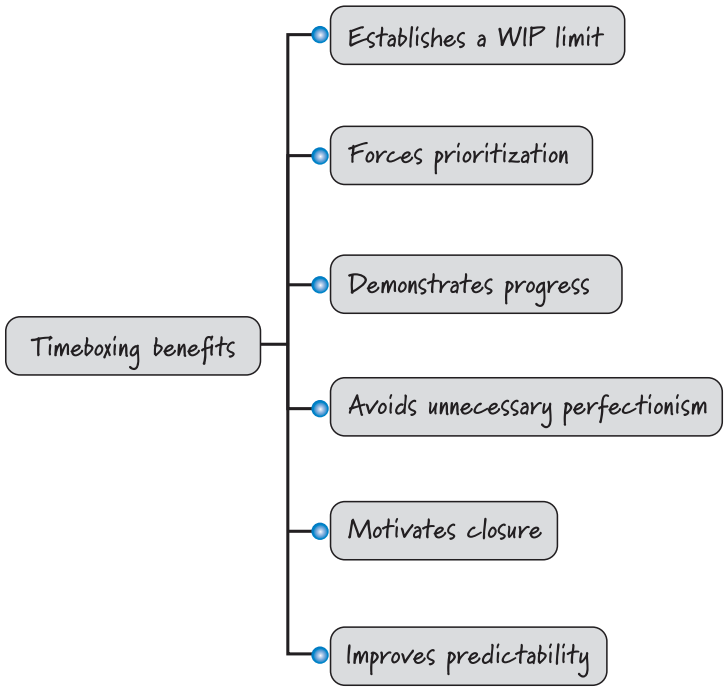


FIGURE 4.2 The benefits of timeboxing

progress reduces organizational risk by shifting the focus away from unreliable forms of progress reporting, such as conformance to plan. Timeboxing also helps us demonstrate progress against big features that require more than one timebox to complete. Completing some work toward those features ensures that valuable, measurable progress is being made each sprint. It also helps the stakeholders and team learn exactly what remains to be done to deliver the entire feature.

Avoids Unnecessary Perfectionism

Timeboxing helps avoid unnecessary perfectionism. At one time or another we have all spent too much time trying to get something "perfect" or to do "gold plating" when "good enough" would suffice. Timeboxing forces an end to potentially unbounded work by establishing a fixed end date for the sprint by which a good solution must be done.

Motivates Closure

Timeboxing also motivates closure. My experience is that things are more likely to get done when teams have a known end date. The fact that the end of the sprint brings with it a hard deadline encourages team members to diligently apply themselves to complete the work on time. Without a known end date, there is less of a sense of urgency to complete the job.

Improves Predictability

Timeboxing improves predictability. Although we can't predict with great certainty exactly the work we will complete a year from now, it is completely reasonable to expect that we can predict the work we can complete in the next short sprint.

Short Duration

Short-duration sprints provide many benefits (see Figure 4.3).

Ease of Planning

Short-duration sprints make it easier to plan. It is easier to plan a few weeks' worth of work than six months' worth of work. Also, planning on such short time horizons requires far less effort and is far more accurate than longer-horizon planning.

Fast Feedback

Short-duration sprints generate fast feedback. During each sprint we create working software and then have the opportunity to inspect and adapt what we built and how we built it. This fast feedback enables us to quickly prune unfavorable product paths or development approaches before we compound a bad decision with many follow-on

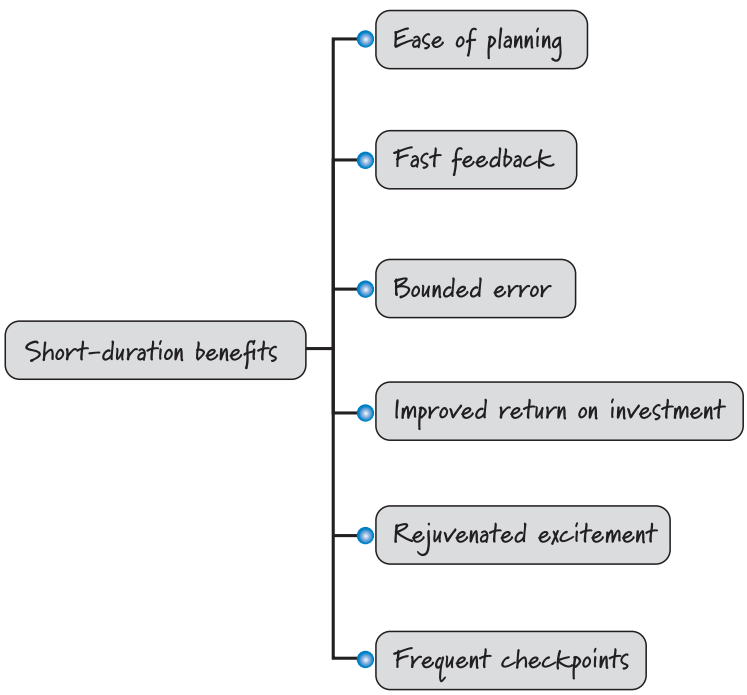


FIGURE 4.3 The benefits of short-duration sprints

decisions that are coupled to the bad decision. Fast feedback also allows us to more quickly uncover and exploit time-sensitive emergent opportunities.

Improved Return on Investment

Short-duration sprints not only improve the economics via fast feedback; they also allow for early and more frequent deliverables. As a result, we have the opportunity to generate revenue sooner, improving the overall return on investment (see Chapter 14 for an example).

Bounded Error

Short-duration sprints also bound error. How wrong can we be in a two-week sprint? Even if we fumble the whole thing, we have lost only two weeks. We insist on short-duration sprints because they provide frequent coordination and feedback. That way, if we're wrong, at least we're wrong in a small way.

Rejuvenated Excitement

Short-duration sprints can help rejuvenate excitement. It is human nature for interest and excitement to decline the longer we have to wait for gratification (see Figure 4.4).

If we work on a very long-duration project, not only are we more likely to fail; we are also more likely to eventually lose enthusiasm for the effort. (When I worked at IBM, we used to call these the "boil-the-ocean" projects, because they would

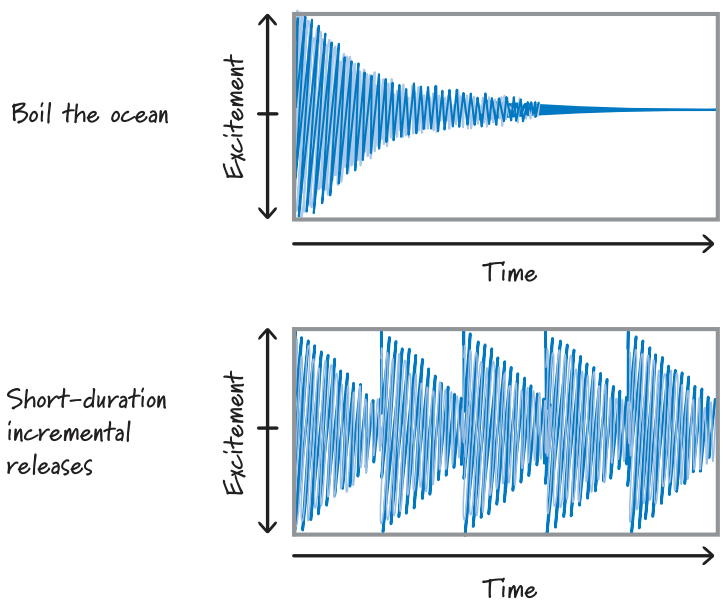


FIGURE 4.4 Excitement over time

take a really long time and a lot of effort to complete, if ever, like trying to boil an ocean.) With no visible progress and no end in sight, people begin to grow disinterested. Toward the end, they may be willing to *pay* someone to get moved to a different product!

Short-duration sprints keep participant excitement high by delivering working assets frequently. The gratification from early and frequent deliverables rejuvenates our interest and our desire to continue working toward the goal.

Frequent Checkpoints

Short-duration sprints also provide multiple, meaningful checkpoints (see Figure 4.5).

One valued aspect of sequential projects is a well-defined set of milestones. These milestones provide managers with known project-lifecycle checkpoints that are usually tied to go/no-go funding decisions for the next phase. Although potentially useful from a governance perspective, as I discussed in Chapter 3 these milestones give an unreliable indication of the true status of customer value delivery.

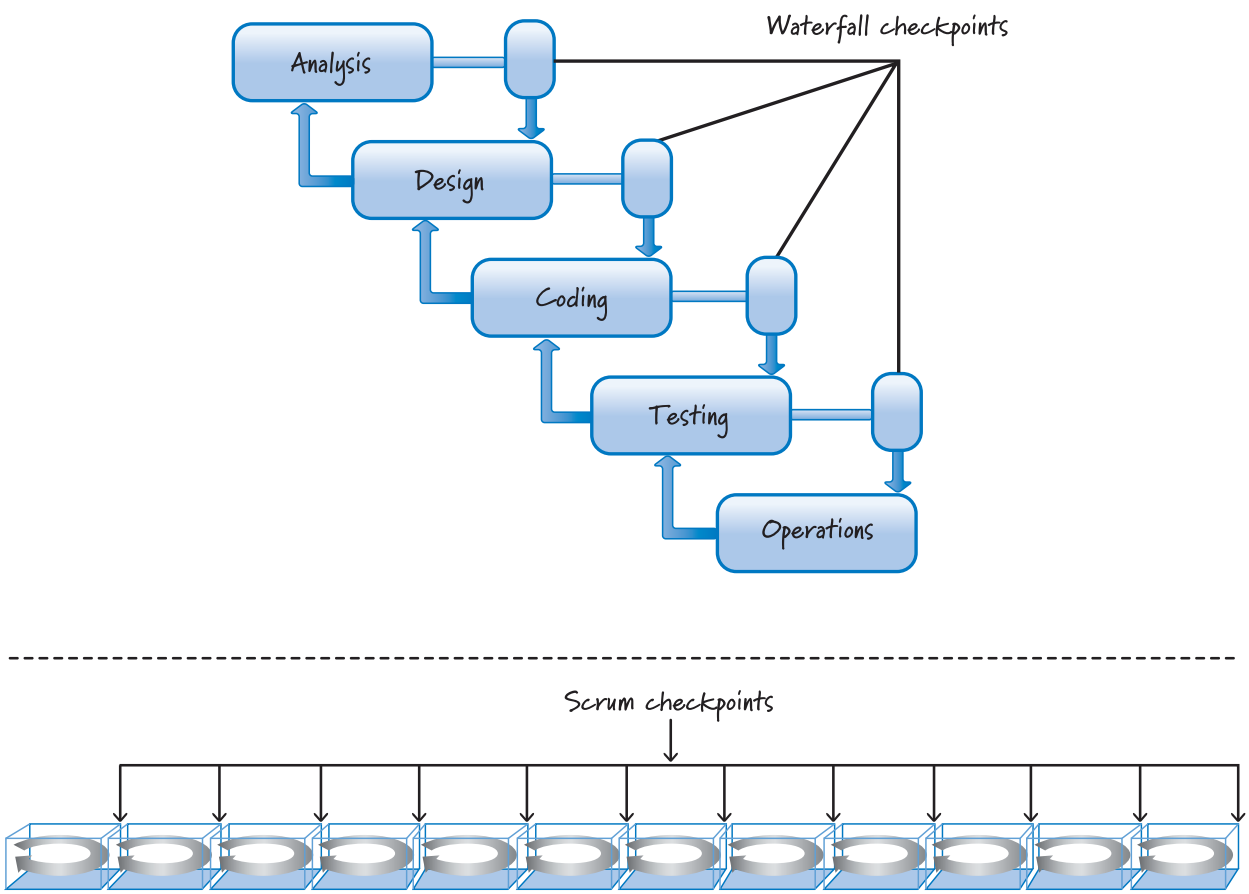


FIGURE 4.5 Checkpoint comparison

Scrum provides managers, stakeholders, product owners, and others with many more checkpoints than they would have with sequential projects. At the end of each short sprint there is a meaningful checkpoint (the sprint review) that allows everyone to base decisions on demonstrable, working features. People are better able to deal with a complex environment when they have more actionable checkpoint opportunities to inspect and adapt.

Consistent Duration

As a rule, on a given development effort, a team should pick a consistent duration for its sprints and not change it unless there is a compelling reason. Compelling reasons might include the following:

- You are considering moving from four-week sprints to two-week sprints in order to obtain more frequent feedback but want to try a couple of two-week sprints before making a final decision.
- The annual holidays or end of the fiscal year make it more practical to run a three-week sprint than the usual two-week sprint.
- The product release occurs in one week, so a two-week sprint would be wasteful.

The fact that the team cannot get all the work done within the current sprint length is not a compelling reason to extend the sprint length. Neither is it permissible to get to the last day of the sprint, realize you are not going to be done, and lobby for an extra day or week. These are symptoms of dysfunction and opportunities for improvement; they are not good reasons to change the sprint length.

As a rule, therefore, if a team agrees to perform two-week sprints, all sprints should be two weeks in duration. As a practical matter, most (but not all) teams will define two weeks to mean ten calendar weekdays. If there is a one-day holiday or training event during the sprint, it reduces the team's capacity for that sprint but doesn't necessitate a sprint length change.

Using the same sprint length also leverages the benefits of cadence and simplifies planning.

Cadence Benefits

Sprints of the same duration provide us with **cadence**—a regular, predictable rhythm or heartbeat to a Scrum development effort. A steady, healthy heartbeat allows the Scrum team and the organization to acquire an important rhythmic familiarity with when things need to happen to achieve the fast, flexible flow of business value. In my experience, having a regular cadence to sprints enables people to "get into the zone," "be on a roll," or "get into a groove." I believe this happens because regular cadence

makes the mundane but necessary activities habitual, thereby freeing up mental capacity to stay focused on the fun, value-added work.

Having a short sprint cadence also tends to level out the intensity of work. Unlike a traditional sequential project where we see a steep increase in intensity in the latter phases, each sprint has an intensity profile that is similar to that of the other sprints. As I will discuss in Chapter 11, sprint cadence enables teams to work at a sustainable pace.

Sprinting on a regular cadence also significantly reduces coordination overhead. With fixed-length sprints we can predictably schedule the sprint-planning, sprint review, and sprint retrospective activities for many sprints at the same time. Because everyone knows when the activities will occur, the overhead required to schedule them for a large batch of sprints is substantially reduced.

As an example, if we do two-week sprints on a yearlong development effort, we can send out the recurring event on everyone's calendar for the next 26 sprint reviews. If we allowed sprint durations to vary from sprint to sprint, imagine the extra effort we would need to coordinate the schedules of the stakeholders on what might be just one or two weeks' notice for an upcoming sprint review! That assumes that we could even find a time that worked for the core set of stakeholders, whose schedules are likely filled up many weeks ahead.

Finally, if we have multiple teams on the same project, having all teams with a similar sprint cadence allows for synchronization of the work across all of the teams (see Chapter 12 for a more detailed discussion).

Simplifies Planning

Using a consistent duration also simplifies planning activities. When all sprints are the same length (even when they might have a day or less capacity per sprint because of a holiday), the team gets comfortable with the amount of work that it can accomplish in a typical sprint (referred to as its **velocity**). Velocity is typically normalized to a sprint. If the length of the sprint can vary, we really don't have a normalized sprint unit. It wouldn't be meaningful to say things like "The team has an average velocity of 20 points per sprint."

While it is certainly possible to compute a team's velocity even if it uses variable-length sprints, it is more complicated. Sticking with a consistent sprint duration simplifies the computations we perform on a team's historical velocity data.

Consistent sprint durations also simplify the rest of the planning math. For example, if we are working on a **fixed-date release**, and we have consistent-duration sprints, calculating the number of sprints in the release is simply an exercise in calendar math (we know today's date, we know the release date, and we know that all sprints are the same length). If the sprint durations were allowed to vary, calculating the number of sprints in the release could be significantly more challenging (because we would have to do extensive early planning), involve unnecessary overhead, and likely be far less reliable than with consistent sprint durations.

No Goal-Altering Changes

An important Scrum rule states that once the sprint goal has been established and sprint execution has begun, no change is permitted that can materially affect the sprint goal.

What Is a Sprint Goal?

Each sprint can be summarized by a sprint goal that describes the business purpose and value of the sprint. Typically the sprint goal has a clear, single focus, such as

- Support initial report generation.
- Load and curate North America map data.
- Demonstrate the ability to send a text message through an integrated software, firmware, and hardware stack.

There are times when a sprint goal might be multifaceted, for example, "Get basic printing working and support search by date."

During sprint planning, the development team should help refine and agree to the sprint goal and use it to determine the product backlog items that it can complete by the end of the sprint (see Chapter 19 for more details). These product backlog items serve to further elaborate the sprint goal.

Mutual Commitment

The sprint goal is the foundation of a mutual commitment made by the team and the product owner. The team commits to meeting the goal by the end of the sprint, and the product owner commits to not altering the goal during the sprint.

This mutual commitment demonstrates the importance of sprints in balancing the needs of the business to be adaptive to change, while allowing the team to concentrate and efficiently apply its talent to create value during a short, fixed duration. By defining and adhering to a sprint goal, the Scrum team is able to stay focused (in the zone) on a well-defined, valuable target.

Change versus Clarification

Although the sprint goal should not be materially *changed*, it is permissible to *clarify* the goal. Let me differentiate the two.

What constitutes a change? A change is any alteration in work or resources that has the potential to generate economically meaningful waste, harmfully disrupt the flow of work, or substantially increase the scope of work within a sprint. Adding or removing a product backlog item from a sprint or significantly altering the scope of a product backlog item that is already in the sprint typically constitutes change. The following example illustrates a change:

> Product owner: "Oh, when I said that we need to be able to search the police database for a juvenile offender, I didn't just mean by last name and first name. I also meant we should be able to search the database based on a picture of the suspect's body tattoos!"

Adding the ability to search based on a picture likely represents substantially more effort and almost certainly would affect the team's ability to meet a commitment to deliver search based on last name and first name. In this case, the product owner should consider creating a new product backlog item that captures the search-by-picture feature and adding it to the product backlog to be worked on in a subsequent sprint.

What constitutes a clarification? Clarifications are additional details provided during the sprint that assist the team in achieving its sprint goal. As I will discuss in Chapter 5, all of the details associated with product backlog items might not be fully known or specified at the start of the sprint. Therefore, it is completely reasonable for the team to ask clarifying questions during a sprint and for the product owner to answer those questions. The following example illustrates a clarification:

> Development team: "When you said the matches for a juvenile offender search should be displayed in a list, did you have a preference for how that list is to be ordered?"
>
> Product owner: "Yes, sort them alphabetically by last name."
>
> Development team: "OK, we can do that."

In this manner, the product owner can and should provide clarification during the sprint.

Consequences of Change

It may appear that the no-goal-altering-change rule is in direct conflict with the core Scrum principle that we should embrace change. We do embrace change, but we want to embrace it in a balanced, economically sensible way.

The economic consequences of a change increase as our level of investment in the changed work increases (see Figure 4.6).

We invest in product backlog items to get them ready to be worked on in a sprint. However, once a sprint starts, our investment in those product backlog items has increased (because we spent time during sprint planning to discuss and plan them at a task level). If we want to make a change after sprint planning has occurred, we not only jeopardize the planning investment, but we also incur additional costs for having to replan any changes during the sprint.

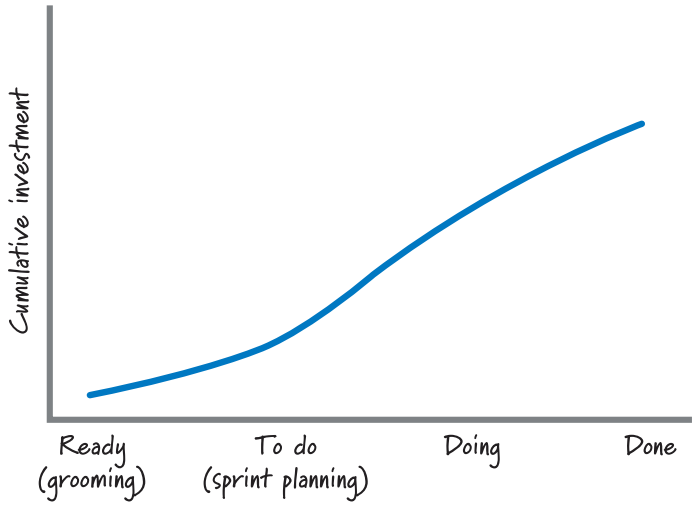


FIGURE 4.6 Cumulative investment at different states

In addition, once we begin sprint execution, our investment in work increases even more as product backlog items transition through the states of to do (work not yet started), doing (work in process), and done (work completed).

Let's say we want to swap out feature X, currently part of the sprint commitment, and substitute feature Y, which isn't part of the existing commitment. Even if we haven't started working on feature X, we still incur planning waste. In addition, feature X might also have dependencies with other features in the sprint, so a change that affects feature X could affect one or more other features, thus amplifying the effect on the sprint goal.

If work on feature X has already begun, in addition to the already-mentioned waste, we could have other potential wastes. For example, all of the work already performed on feature X might have to be thrown away. And we might have the additional waste of removing the partially completed work on feature X, which we may never use in the future (we're not going to include partially completed work in a potentially shippable product increment at the end of the sprint).

And, of course, if feature X is already completed, we might have wasted the full investment we made in feature X. All of this waste adds up!

In addition to the direct economic consequences of waste, the economics can be indirectly affected by the potential deterioration of team motivation and trust that can accompany a change. When the product owner makes a commitment to not alter the goal and then violates the commitment, the team naturally will be demotivated, which will almost certainly affect its desire to work diligently to complete other product backlog items. In addition, violating the commitment can harm the trust within the Scrum team, because the development team will not trust that the product owner is willing to stick to his commitments.

Being Pragmatic

The no-goal-altering-change rule is just that—a rule, not a law. The Scrum team has to be pragmatic.

What if business conditions change in such a way that making a change to the sprint goal seems warranted? Say a competitor launches its new product during our sprint. After reviewing the new product, we conclude that we need to alter the goal we established for our current sprint because what we are doing is now economically far less valuable given what our competitor has done. Should we blindly follow the rule of no goal-altering changes and not alter our sprint? Probably not.

What if a critical production system has failed miserably and some or all of the people on our team are the only ones who can fix it? Should we not interrupt the current sprint to fix it? Do we tell the business that we will fix the production failure first thing next sprint? Probably not.

In the end, being pragmatic trumps the no-goal-altering-change rule. We must act in an economically sensible way. Everyone on the Scrum team can appreciate that. If we change the current sprint, we will experience the negative economic consequences I previously discussed. However, if the economic consequences of the change are far less than the economic consequences of deferring the change, making the change is the smart business decision. If the economics of changing versus not changing are immaterial, no change to the sprint goal should be made.

As for team motivation and trust, in my experience, when a product owner has a frank, economically focused discussion with the team about the necessity of the change, most teams understand and appreciate the need, so the integrity of motivation and trust is upheld.

Abnormal Termination

Should the sprint goal become completely invalid, the Scrum team may decide that continuing with the current sprint makes no sense and advise the product owner to abnormally terminate the sprint. When a sprint is abnormally terminated, the current sprint comes to an abrupt end and the Scrum team gathers to perform a sprint retrospective. The team then meets with the product owner to plan the next sprint, with a different goal and a different set of product backlog items.

Sprint termination is used when an economically significant event has occurred, such as a competitor's actions that completely invalidate the sprint or product funding being materially changed.

Although the product owner reserves the option to cancel each and every sprint, in my experience it is rare that product owners invoke this option. Often there are less drastic measures that a Scrum team can take to adjust to the situation at hand. Remember, sprints are short, and, on average, the team will be about halfway through a sprint when a change-causing situation arises. Because there may be only a week or so of time left in the sprint when the change occurs, the economics of terminating may be less favorable than just staying the course. And many times it is possible to

make a less dramatic change, such as dropping a feature to allow time to fix a critical production failure, instead of terminating the sprint.

It is important to realize that terminating the sprint early, in addition to having a negative effect on morale, is a serious disruption of the fast, flexible flow of features and negates many of the benefits of consistent-duration sprints I mentioned earlier. Terminating a sprint should be the last resort.

If a sprint is terminated, the Scrum team will have to determine the length of the next sprint (see Figure 4.7).

There are three apparent options:

1. Stay with the original sprint length. This has the advantage of keeping a uniform sprint length throughout development (except for the terminated sprint, of course). If multiple Scrum teams are collaborating on the same development effort, using the original sprint length will put the Scrum team that terminated its sprint out of sync with the other teams.
2. Make the next sprint just long enough to get to the end date of the terminated sprint. For example, if the Scrum team terminated a two-week sprint at the end of the first week, the next sprint would be one week to get the team resynchronized to its original sprint cadence.
3. Make the next sprint bigger than a normal sprint to cover the remaining time in the terminated sprint plus the time for the next full sprint. So, in the previous example, make the next sprint three weeks in order to get the team resynchronized to its original sprint cadence.

In a multiteam development effort, either option 2 or option 3 would be preferred. In all cases, you will need to consider your specific context to know which option is best.

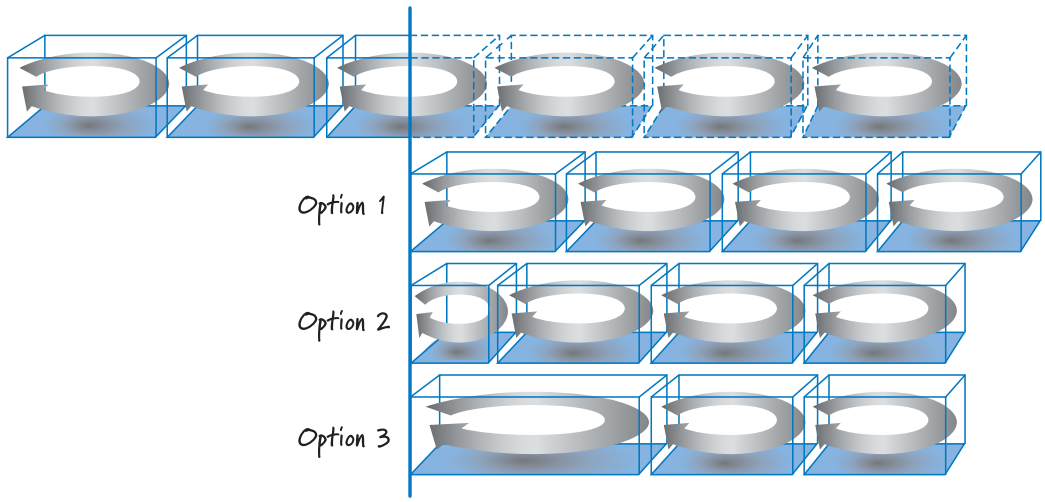


FIGURE 4.7 Deciding on the next sprint length after sprint termination

Definition of Done

In Chapter 2, I discussed how the result of each sprint should be a potentially shippable product increment. I also mentioned that "potentially shippable" doesn't mean that what was built must actually be shipped. Shipping is a business decision that often occurs at a different cadence; in some organizations it may not make sense to ship at the end of every sprint.

Potentially shippable is better thought of as a state of confidence that what got built in the sprint is actually done, meaning that there isn't materially important undone work (such as important testing or integration and so on) that needs to be completed before we could ship the results from the sprint, if shipping was our business desire. To determine if what got produced is potentially shippable, the Scrum team must have a well-defined, agreed-upon definition of done.

What Is the Definition of Done?

Conceptually the **definition of done** is a checklist of the types of work that the team is expected to successfully complete before it can declare its work to be potentially shippable (see Table 4.1).

TABLE 4.1 Example Definition-of-Done Checklist

	Definition of Done
❑	Design reviewed
❑	Code completed
❑	Code refactored
❑	Code in standard format
❑	Code is commented
❑	Code checked in
❑	Code inspected
❑	End-user documentation updated
❑	Tested
❑	Unit tested
❑	Integration tested
❑	Regression tested
❑	Platform tested
❑	Language tested
❑	Zero known defects
❑	Acceptance tested
❑	Live on production servers

Obviously the specific items on the checklist will depend on a number of variables:

- The nature of the product being built
- The technologies being used to build it
- The organization that is building it
- The current impediments that affect what is possible

Most of the time, a bare-minimum definition of done should yield a complete slice of product functionality, one that has been designed, built, integrated, tested, and documented and would deliver validated customer value. To have a useful checklist, however, these larger-level work items need to be further refined. For example, what does tested mean? Unit tested? Integration tested? System tested? Platform tested? Internationalization tested? You can probably think of many other forms of testing that are specific to your product. Are all of those types of testing included in the definition of done?

Keep in mind that if you don't do an important type of testing every sprint (say, performance testing), you'll have to do it sometime. Are you going to have some specialized sprint in the future where the only thing you do is performance testing? If so, and performance testing is essential to being "done," you really don't have a potentially shippable product increment each sprint. And even worse, when you actually do the performance testing at a later time and it doesn't go quite as planned, not only will you discover a critical problem very late in the process, but you will also have to spend much more time and money to fix it at that time than if you had done the performance testing earlier.

Sometimes the testing might take longer than the duration of a sprint. If this occurs because the development team has accrued a huge manual testing debt, the team needs to start automating its tests so that the testing can be completed within a sprint. If this occurs because of the nature of the test, we will need to accept starting the test in one sprint and finishing it in some future sprint. For example, one organization I coached was building a device composed of hardware, firmware, and software. One of its standard tests was the 1,500-hour burn-in test, where the device was run flat-out for that amount of time to see if it would fail. That test can't be completed in a two-week sprint, so the Scrum team adjusted the definition of done so that a sprint could be deemed done even if the 1,500-hour test was not yet completed.

Often I am asked, "What if there is a significant defect that remains on the last day of the sprint; is the product backlog item done?" No, it's not done! And because, as a rule, we don't extend sprints beyond the end of the planned timebox, we wouldn't extend the sprint by a day or two to fix the defect in the current sprint. Instead, at the planned end of the sprint, the incomplete product backlog item is taken from the current sprint and reinserted into the product backlog in the proper order based on the other items that are currently in the product backlog. The incomplete item might then be finished in some future sprint.